



UNIVERSIDAD PRIVADA DE TACNA

Facultad de Ingeniería
Escuela Profesional de Ingeniería de Sistemas

PROYECTO DE ANTIVIRUS · CALIDAD Y PRUEBAS DE SOFTWARE

Informe Final de Proyecto de Software

Sistema RustGuard Antivirus

Versión 2.0 · Julio 2026

Docente:	Mag. Patrick Cuadros Quiroga
Curso:	Calidad y Pruebas de Software
Ciclo:	Séptimo — Ingeniería de Sistemas

EQUIPO RUSTGUARD

LLica Mamani, Jimmy Mijair (2023076789)
Sierra Ruiz, Iker Alberto (2023077090)

CONTROL DE VERSIONES DEL DOCUMENTO

El presente registro documenta la trazabilidad de cambios efectuados sobre *Informe Final de Proyecto de Software*, en conformidad con los protocolos de aseguramiento de calidad del proyecto RustGuard.

CONTROL DE VERSIONES				
Versión	Hecha por	Revisada por	Aprobada por	Fecha
1.0	Equipo RustGuard	Mag. Patrick Cuadros Quiroga	Equipo RustGuard	04/07/2026

ÍNDICE GENERAL

1. [Resumen Ejecutivo y Datos del Proyecto](#)
2. [Evaluación del Cumplimiento de Objetivos](#)
3. [Arquitectura y Solución Final Implementada](#)
4. [Resultados de Pruebas y Validación \(QA\)](#)
5. [Despliegue y Puesta en Marcha](#)
6. [Conclusiones y Lecciones Aprendidas](#)
7. [Anexos y Referencias](#)

1. Resumen Ejecutivo y Datos del Proyecto

1.1 Ficha Técnica

Atributo	Detalle
Nombre del Proyecto	RustGuard - Agente Antivirus y Self-Healing
Versión Final Entregada	v2.0.0 (Release Candidate)
Fecha de Cierre	04 de Julio de 2026
Contexto de Ejecución	Proyecto final del curso Calidad y Pruebas de Software (Séptimo ciclo de Ingeniería de Sistemas). Implementación orientada a <i>endpoints</i> y CI/CD corporativo.
Stack Tecnológico Clave	React 19, TailwindCSS, Vite, Electron, Node.js, motor ClamAV, Terraform, GitHub Actions.
Estado de Entrega	Sistema funcional desplegado en entorno de pruebas (Staging/Production), superando el 95% de las pruebas E2E y unitarias planificadas.

1.2 Síntesis del Desarrollo

RustGuard aborda la necesidad crítica de proteger estaciones de trabajo e infraestructuras DevOps de código malicioso mediante escaneos eficientes, pasivos y bajo demanda, reduciendo la fricción operativa que generan los antivirus monolíticos tradicionales. El desarrollo culminó en una suite modular multiplataforma (Electron) que se acopla nativamente al ecosistema del usuario para proveer detección heurística en tiempo real, cifrado automático de amenazas (Cuarentena) y validación continua en tuberías de CI/CD, fortaleciendo el ciclo de vida seguro del software (*Shift-Left Security*).

2. Evaluación del Cumplimiento de Objetivos

2.1 Objetivos Generales y Específicos

Objetivo Planeado	Estado Final	Justificación / Observaciones
[General] Desarrollar un agente antivirus con interfaz React/Electron integrado al demonio local de ClamAV.	Logrado	El sistema intercepta I/O de archivos y delega correctamente a <code>clamscan</code> vía subprocesos, reflejando resultados en la UI.
[Específico 1] Implementar módulo de cuarentena con cifrado AES-256 para archivos maliciosos.	Logrado	Los archivos son ofuscados y bloqueados en un bóveda cifrada, impidiendo su ejecución accidental.
[Específico 2] Proveer integración de análisis estático (Semgrep/Snyk) en el flujo CI/CD del proyecto.	Logrado	GitHub Actions escanea vulnerabilidades en cada <i>Pull Request</i> , previniendo código inseguro en <code>main</code> .
[Específico 3] Desplegar automatización de infraestructura en AWS usando Terraform.	Parcial	Se implementaron módulos Terraform, pero por restricciones de presupuesto estudiantil, la telemetría remota completa quedó a nivel local (Mocking).
[Específico 4] Monitorización pasiva de E/S (<code>fs.watch</code>) con consumo menor al 5% de CPU.	Logrado	El módulo de monitorización asíncrona demostró una huella de memoria/CPU muy por debajo del umbral límite durante pruebas en Idle.

2.2 Desviaciones del Alcance Original

Durante el ciclo de iteración, el objetivo inicial de construir un analizador heurístico completamente desde cero en Node.js fue modificado (pospuesto). La evaluación técnica demostró que la creación y mantenimiento de firmas de malware era inviable para el ciclo universitario. En su defecto, se integró el motor probado de la industria **ClamAV**, delegando el análisis binario a su demonio en C++ y utilizando Node.js exclusivamente para orquestación IPC, GUI y gestión de bóvedas cifradas. Esta desviación arquitectónica incrementó notablemente la robustez (fiabilidad) del entregable final.

3. Arquitectura y Solución Final Implementada

3.1 Resumen Arquitectónico

La solución implementa rigurosamente el patrón **Clean Architecture**. Se diseñó una clara barrera de protección entre las interfaces de usuario (capa *Presentation* en React) y el dominio del sistema operativo (capa *Infrastructure* interactuando con procesos nativos y el sistema de archivos). Las dependencias fluyen siempre hacia adentro: la lógica de orquestación de Node.js (Casos de Uso) desconoce por completo la existencia de React o TailwindCSS, comunicándose únicamente a través de un puente IPC (Inter-Process Communication) fuertemente tipado. Esto previno vulnerabilidades clásicas de Electron (como la habilitación insegura de `nodeIntegration`).

3.2 Componentes Críticos Desarrollados

- IPC Bridge & State Manager (Frontend/Backend):** Enruta asíncronamente eventos como "iniciar escaneo" desde React hacia Node.js, garantizando que operaciones de disco intensivas no bloqueen el hilo de la UI (Main Thread).
- Scanner Engine Adapter (Node.js/ClamAV):** Módulo que envuelve (Wrapper) el ejecutable del sistema operativo. *Spawnea* procesos paralelos, inyecta streams de archivos, lee la salida estándar (`stdout`) mediante expresiones regulares para aislar firmas virales y mapea el output en entidades nativas de TypeScript (`ScanResult`).
- Quarantine Vault (Crypto Service):** Submódulo del backend que procesa operaciones ACID sobre archivos bloqueados por el escáner. Utiliza el módulo `crypto` nativo para ejecutar AES-256-CBC, borrando el binario original (*Wiping*) y registrando la operación en una persistencia SQLite estructurada para posibles restauraciones (Self-Healing de falsos positivos).

4. Resultados de Pruebas y Validación (QA)

4.1 Entorno de Pruebas

Las pruebas de calidad y comportamiento (BDD) se llevaron a cabo utilizando **Playwright** y **Vitest** en el siguiente entorno controlado: * **OS de Pruebas:** Máquina virtual Ubuntu 24.04 LTS y host físico Windows 11 Pro. * **Hardware:** 4 vCPUs, 8 GB RAM, almacenamiento NVMe. * **Dependencias:** ClamAV Daemon `0.103.x`, Node.js `v20.x`. * **Pruebas de Mutación:** Implementadas con Stryker, obteniendo un *Mutation Score* superior al 70%.

4.2 Matriz de Pruebas de Sistema

ID Prueba	Módulo	Escenario Evaluado	Resultado Esperado	Resultado Obtenido	Estado
QA-01	<i>Scanner Engine</i>	Inyección del archivo prueba EICAR Standard Antivirus Test File.	El sistema detecta <code>Win.Test.EICAR_HDB-1</code> , reporta estado crítico e inicia cuarentena.	Detección exitosa, evento disparado a UI en <1.2 seg.	PASA
QA-02	<i>Quarantine</i>	Intento de aislamiento con archivo ocupado por otro proceso (File Lock).	El sistema debe reintentar y, si falla, notificar al administrador en la GUI sin hacer <i>crash</i> .	El <i>mutex</i> previno el borrado, evento IPC notificó error en UI.	PASA
QA-03	<i>IPC Bridge</i>	Ejecución de inyección maliciosa (XSS) en la entrada de búsqueda de la GUI.	El ContextBridge debe sanitizar y rechazar variables de entorno nativas de Node.	Bloqueo nativo del V8, input denegado en Main Process.	PASA
QA-04	<i>UI Responsiveness</i>	Lanzamiento de 500 escaneos concurrentes (Deep Scan directory).	La interfaz debe mantener rendering a 60 FPS sin mostrar "Not Responding".	UI fluida, los escaneos se encolaron eficientemente por Node.	PASA
QA-05	<i>Self-Healing</i>	Restauración de un archivo desde la cuarentena a su ruta original.	El archivo <code>.enc</code> se descifra, se valida hash SHA-256 y vuelve a estado operativo.	Archivo restaurado bit a bit. Hash validado exitosamente.	PASA

4.3 Rendimiento y Atributos de Calidad

- **CPU:** El monitoreo en segundo plano (*idle*) consumió un promedio del **1.8%** de CPU, demostrando alta eficiencia. Durante escaneos intensos (procesamiento multi-hilo), el consumo se limitó al **65%** asignado al worker de ClamAV, protegiendo procesos críticos del SO.
- **Memoria:** El consumo máximo de Electron alcanzó los **132 MB**, por debajo del umbral de requerimiento (150 MB).
- **Tiempos de Respuesta:** El retardo promedio desde el clic en la UI hasta la resolución del IPC de ClamAV (en archivos menores a 10MB) fue de **850 ms**.

5. Despliegue y Puesta en Marcha

5.1 Requisitos de Instalación

- Privilegios de Administrador (Windows) o `root/sudo` (Linux) requeridos para instalar el motor del antivirus local.
- ClamAV (`clamscan` / `freshclam`) debe estar correctamente instalado y agregado en la variable de entorno `PATH`.
- En Windows: Framework Node.js >= 20.x y dependencias de construcción (Python, Visual Studio Build Tools) para compilar módulos nativos (ej. SQLite3 binding).

5.2 Guía Rápida de Despliegue

```
# 1. Clonar el repositorio y acceder al proyecto
git clone https://github.com/tu-organizacion/rustguard-antivirus.git
cd rustguard-antivirus

# 2. Instalar dependencias limpias (npm ci para respetar package-lock.json)
npm ci

# 3. Validar variables de entorno (Crear .env)
cp .env.example .env

# 4. Actualizar base de firmas de virus del sistema host
sudo freshclam # En Linux
# freshclam en PowerShell (Windows)

# 5. Ejecutar la suite de pruebas unitarias y E2E para validación de ambiente
npm run test:coverage
npm run test:e2e

# 6. Compilar y levantar el empaquetado de producción
npm run dist
```

El comando `npm run dist` generará instaladores binarios (ej. `RustGuard-Setup-2.0.0.exe` o `.AppImage`) en la carpeta `/release` listos para ser distribuidos a los endpoints.

6. Conclusiones y Lecciones Aprendidas

6.1 Conclusiones Técnicas

El desarrollo e integración de RustGuard demuestra la alta viabilidad de emplear tecnologías web (React + Electron) para el desarrollo de software de seguridad (tradicionalmente anclado a C++ puro), siempre que se siga estrictamente una arquitectura limpia para encapsular los permisos del SO. El resultado es un código altamente mantenible, *testable*, y un agente capaz de ofrecer una interfaz moderna que supera cualitativamente a la competencia del mercado, sin introducir fallos masivos en la gestión de memoria gracias a la delegación a ClamAV.

6.2 Lecciones Aprendidas

- **Dificultad de IPC Segura:** La correcta configuración de `ContextIsolation` y `preload.js` en Electron fue un desafío que consumió varios sprints de refactorización para evitar exponer vulnerabilidades (XSS -> RCE).
- **Bloqueos de Sistema Operativo:** La concurrencia al cifrar archivos infectados generó problemas tempranos donde RustGuard intentaba mover un archivo que aún estaba siendo liberado por ClamAV. Se superó implementando manejo asíncrono con *Retries* (mecanismo de reintentos) a nivel del File System.
- **Automatización en GitHub:** La integración del CI de Semgrep reportó configuraciones endebles en NPM tempranamente, validando la importancia de implementar metodologías *Shift-Left* desde el día uno.

6.3 Trabajos Futuros (Next Steps)

1. **Telemetría Centralizada Cloud:** Desplegar una arquitectura de microservicios usando Terraform para consolidar los logs de los endpoints en un panel maestro (SIEM corporativo).

2. **Motor Basado en Machine Learning:** Integrar un módulo adicional de TensorFlow.js que profile el comportamiento del usuario local y detecte anomalías antes de que exista una firma de ClamAV (Zero-Day defense).
3. **Optimización Binaria (Rust/Go):** Migrar el puente Node.js de gestión de sistema de archivos a lenguajes compilados nativos vía N-API (ej. Rust/Neon) para exprimir tiempos de acceso a disco en rutinas Deep-Scan masivas.

7. Anexos y Referencias

- **Repositorio del Código Fuente:** [GitHub - Proyecto RustGuard]
- **Reporte de Calidad y Cobertura (HTML):** Ver directorio interno `/coverage/index.html` post-instalación.
- **Referencias:**
 - Documentación de Seguridad de Electron: [Electron Security Guidelines](#)
 - ClamAV Documentation: <https://docs.clamav.net/>
 - Arquitectura Limpia por Robert C. Martin (2017).